Atelier 1 : SQL complet

Requêtes, calculs, optimisation et sécurité

Thème : gestion commerciale
(clients, commandes, produits, paiements)

Base de données : **atelier_commerce**
Serveur : **MySQL 8.4.11 hébergé sur Aiven**
Client : **DBeaver**

Sommaire

1. Connexion au serveur MySQL distant (Aiven)
2. Création de la base et des tables
3. Insertion des données
4. Jointure (INNER JOIN)
5. Agrégation (GROUP BY et SUM)
6. Fonction IF()
7. Index et EXPLAIN
8. Vue vue_ventes
9. Gestion des droits (GRANT, SHOW GRANTS, REVOKE)

1. Connexion au serveur MySQL distant (Aiven)

Un service **MySQL 8.4.11** nommé **atelier-mysql** a été créé sur Aiven. Son état est **Running**. La connexion depuis DBeaver (hôte Aiven, port 14691, base defaultdb) a réussi.

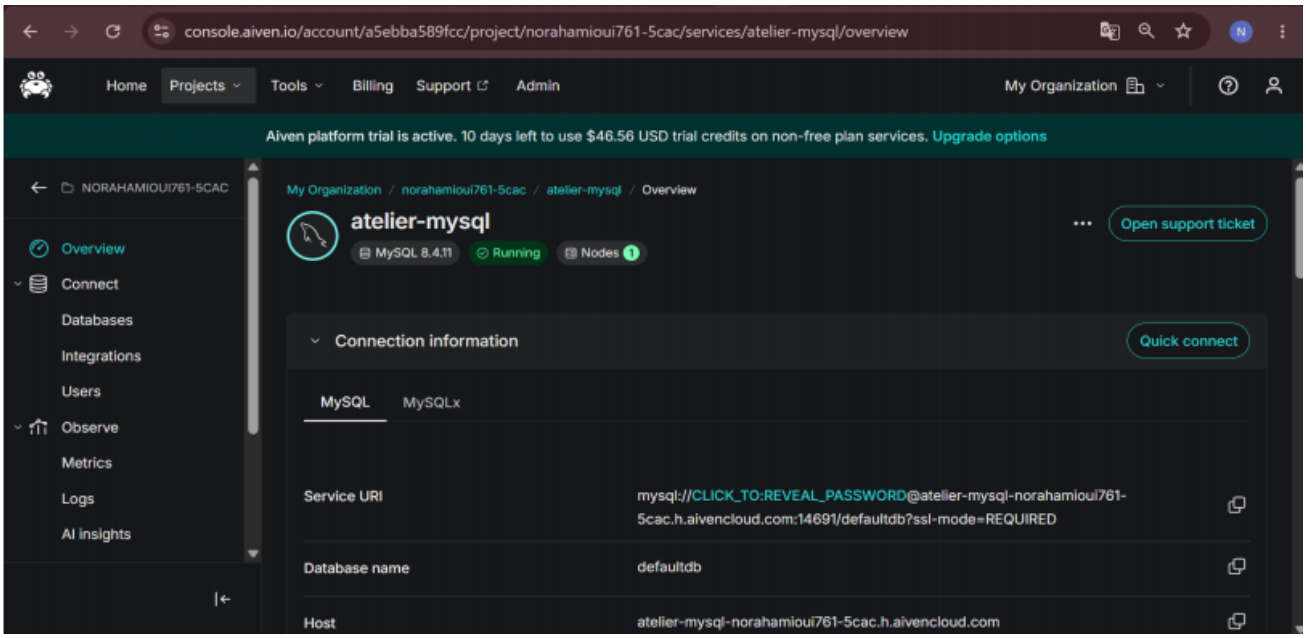


Figure 1 : tableau de bord Aiven, service MySQL en état Running

2. Création de la base et des tables

La base **atelier_commerce** puis les 5 tables (clients, produits, commandes, details_commandes, paiements) ont été créées. Les 6 requêtes se sont exécutées sans erreur.

```
CREATE DATABASE atelier_commerce;
USE atelier_commerce;

CREATE TABLE clients (
  id INT AUTO_INCREMENT PRIMARY KEY, nom VARCHAR(50), ville VARCHAR(50));

CREATE TABLE produits (
  id INT AUTO_INCREMENT PRIMARY KEY, nom_produit VARCHAR(50), prix_unitaire DECIMAL(10,2));

CREATE TABLE commandes (
  id INT AUTO_INCREMENT PRIMARY KEY, client_id INT, date_commande DATE,
  FOREIGN KEY (client_id) REFERENCES clients(id));

CREATE TABLE details_commandes (
  id INT AUTO_INCREMENT PRIMARY KEY, commande_id INT, produit_id INT, quantite INT,
  FOREIGN KEY (commande_id) REFERENCES commandes(id),
  FOREIGN KEY (produit_id) REFERENCES produits(id));

CREATE TABLE paiements (
  id INT AUTO_INCREMENT PRIMARY KEY, commande_id INT, montant DECIMAL(10,2),
  mode_paiement VARCHAR(30), date_paiement DATE,
  FOREIGN KEY (commande_id) REFERENCES commandes(id));
```

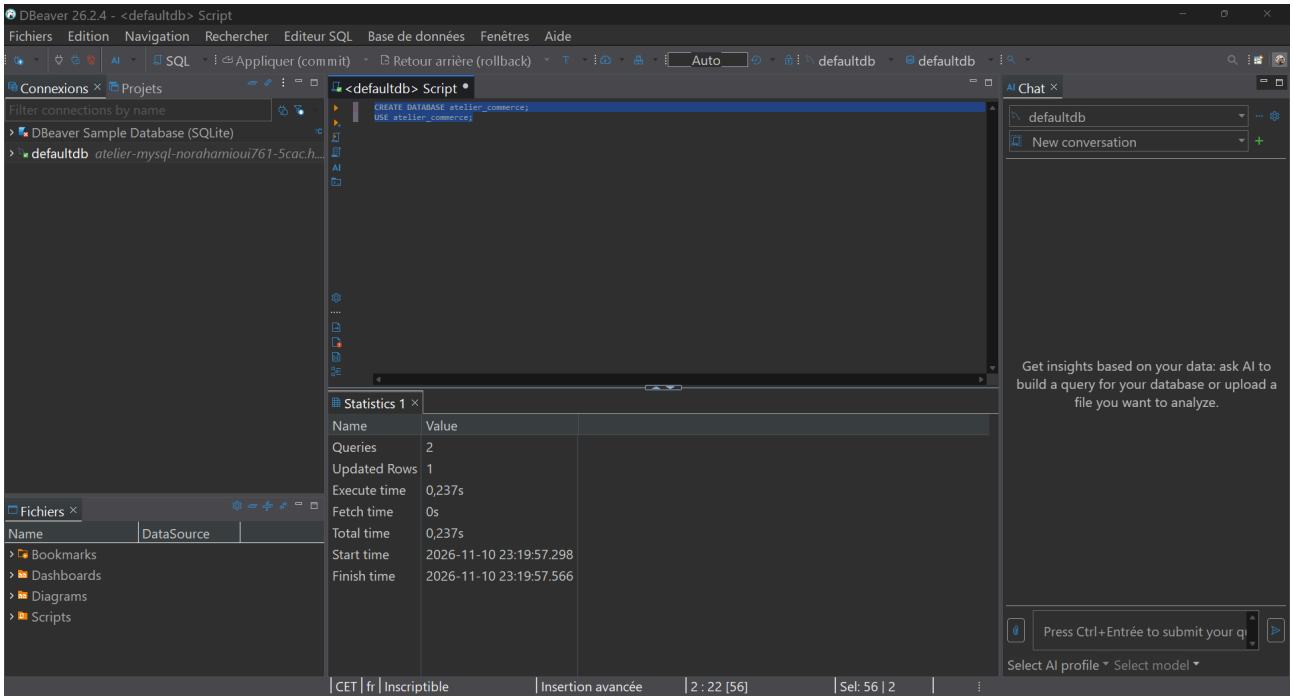


Figure 2 : création de la base atelier_commerce

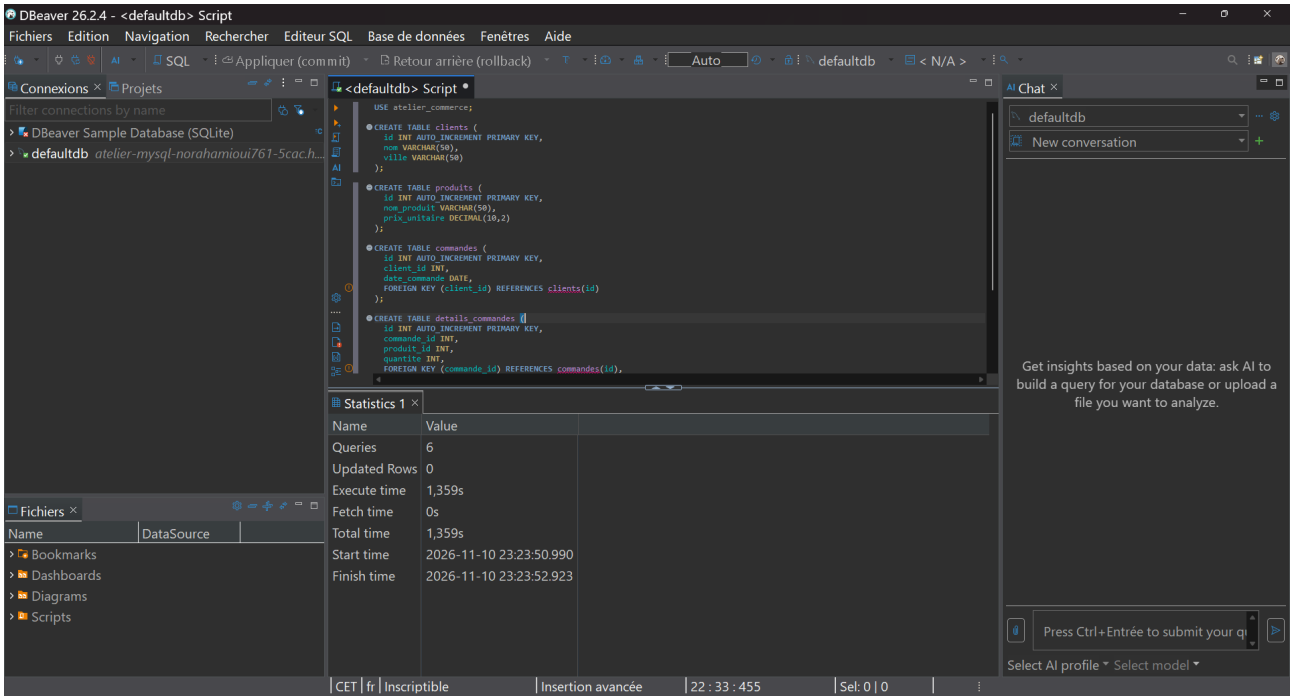


Figure 3 : création des tables (Queries : 6)

3. Insertion des données

Les données de l'atelier ont été insérées : 20 lignes au total (4 + 4 + 4 + 5 + 3).

```
INSERT INTO clients (nom, ville)
VALUES ('Alice','Paris'), ('Bob','Lyon'), ('Charlie','Toulouse'), ('Dina','Nice');

INSERT INTO produits (nom_produit, prix_unitaire)
VALUES ('Ordinateur',950), ('Souris',20), ('Clavier',45), ('Écran',250);

INSERT INTO commandes (client_id, date_commande)
VALUES (1,'2025-01-10'), (1,'2025-01-20'), (2,'2025-02-01'), (3,'2025-02-10');

INSERT INTO details_commandes (commande_id, produit_id, quantite)
VALUES (1,1,1), (1,2,2), (2,3,1), (3,4,2), (4,2,3);

INSERT INTO paiements (commande_id, montant, mode_paiement, date_paiement)
VALUES (1,990,'Carte','2025-01-12'), (2,45,'Espèces','2025-01-21'),
       (3,500,'Virement','2025-02-05');
```

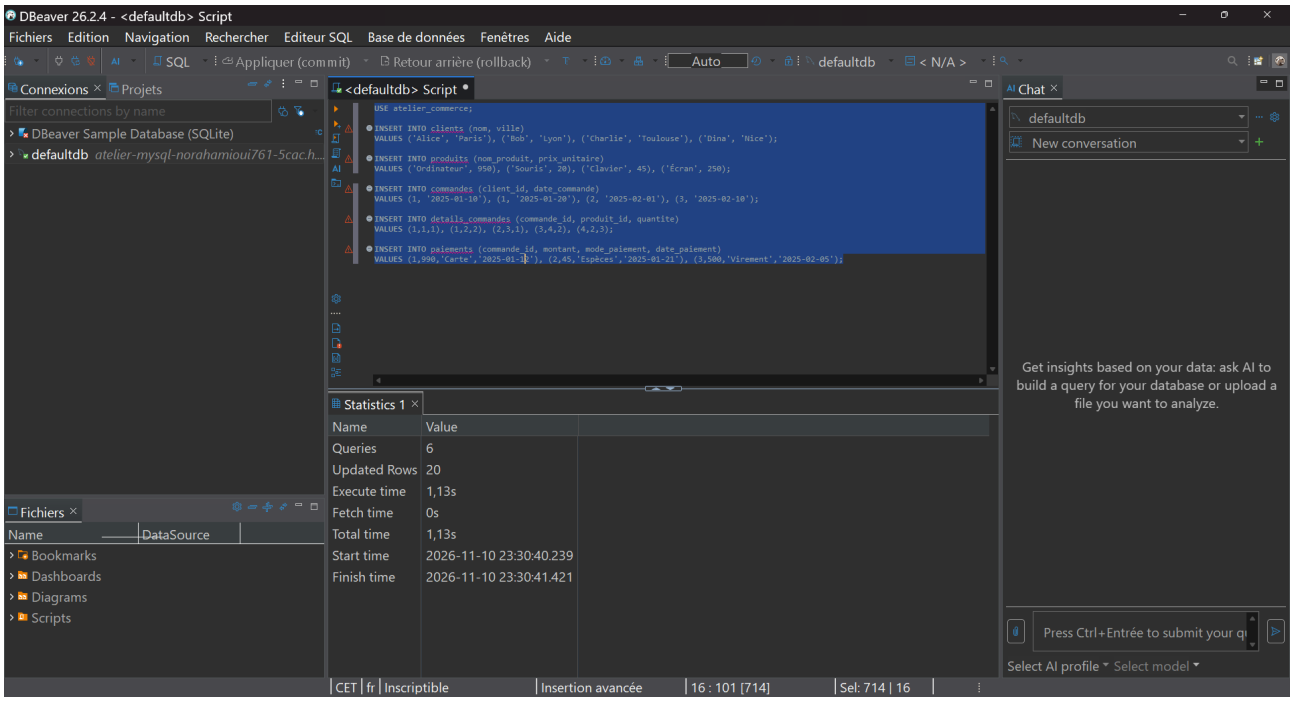


Figure 4 : insertion des données (Updated Rows : 20)

4. Jointure (INNER JOIN)

Cette requête relie les 4 tables pour afficher, pour chaque ligne de commande, le client, le numéro de commande, le produit et la quantité. Elle retourne 5 lignes.

```
SELECT c.nom, cmd.id AS commande, p.nom_produit, dc.quantite
FROM clients c
INNER JOIN commandes cmd ON c.id = cmd.client_id
INNER JOIN details_commandes dc ON cmd.id = dc.commande_id
INNER JOIN produits p ON dc.produit_id = p.id;
```

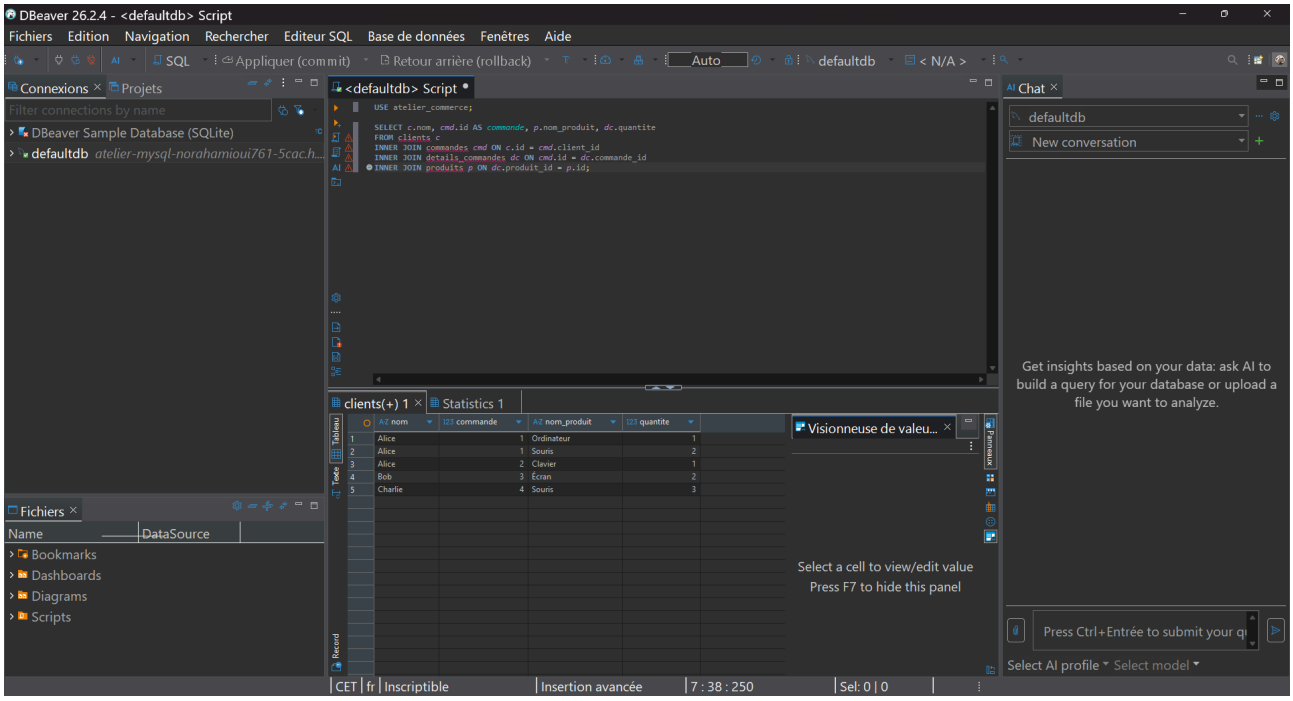


Figure 5 : résultat de l'INNER JOIN

5. Agrégation (GROUP BY et SUM)

Calcul du montant total de chaque commande : commande 1 = 990, commande 2 = 45, commande 3 = 500, commande 4 = 60.

```
SELECT cmd.id AS commande, SUM(p.prix_unitaire * dc.quantite) AS total_commande
FROM commandes cmd
JOIN details_commandes dc ON cmd.id = dc.commande_id
JOIN produits p ON dc.produit_id = p.id
GROUP BY cmd.id;
```

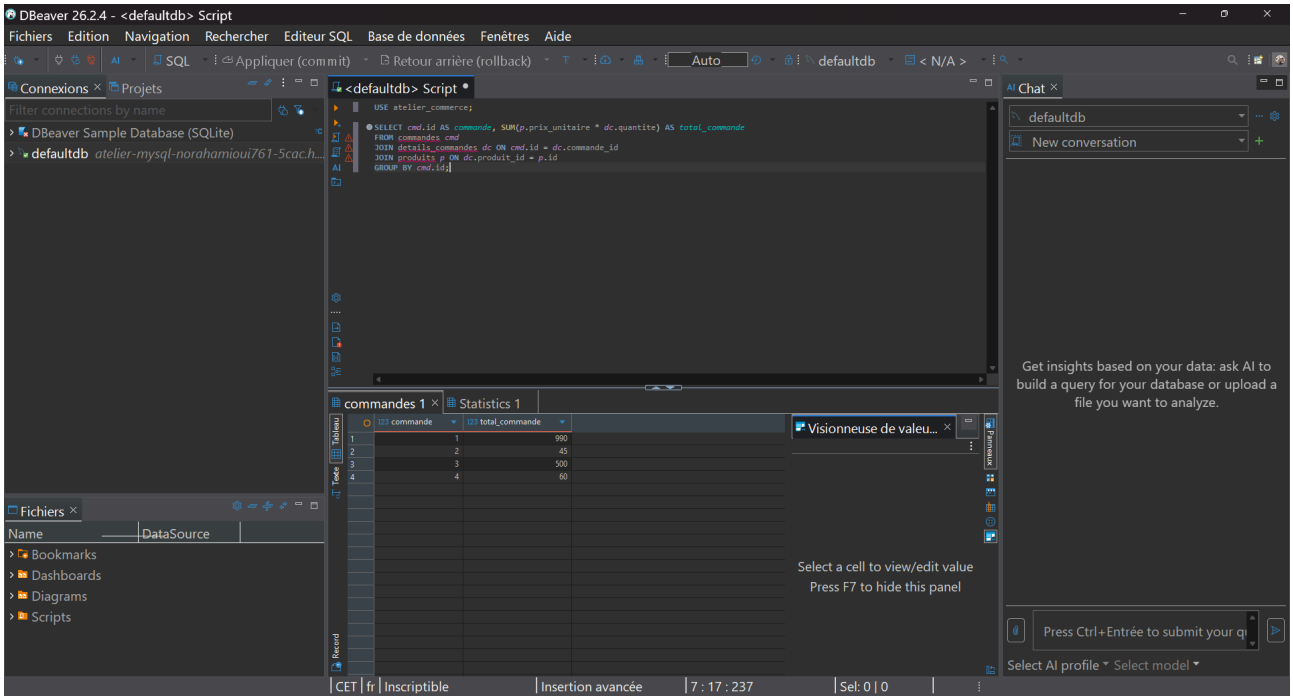


Figure 6 : montant total par commande (GROUP BY + SUM)

6. Fonction IF()

Les clients sont classés selon leur total dépensé : plus de 800 = Premium, sinon Standard. Alice (1035) est Premium, Bob (500) et Charlie (60) sont Standard.

```
SELECT c.nom, SUM(p.prix_unitaire * dc.quantite) AS total,
       IF(SUM(p.prix_unitaire * dc.quantite) > 800, 'Premium', 'Standard') AS categorie
FROM clients c
JOIN commandes cmd ON c.id = cmd.client_id
JOIN details_commandes dc ON cmd.id = dc.commande_id
JOIN produits p ON dc.produit_id = p.id
GROUP BY c.nom;
```

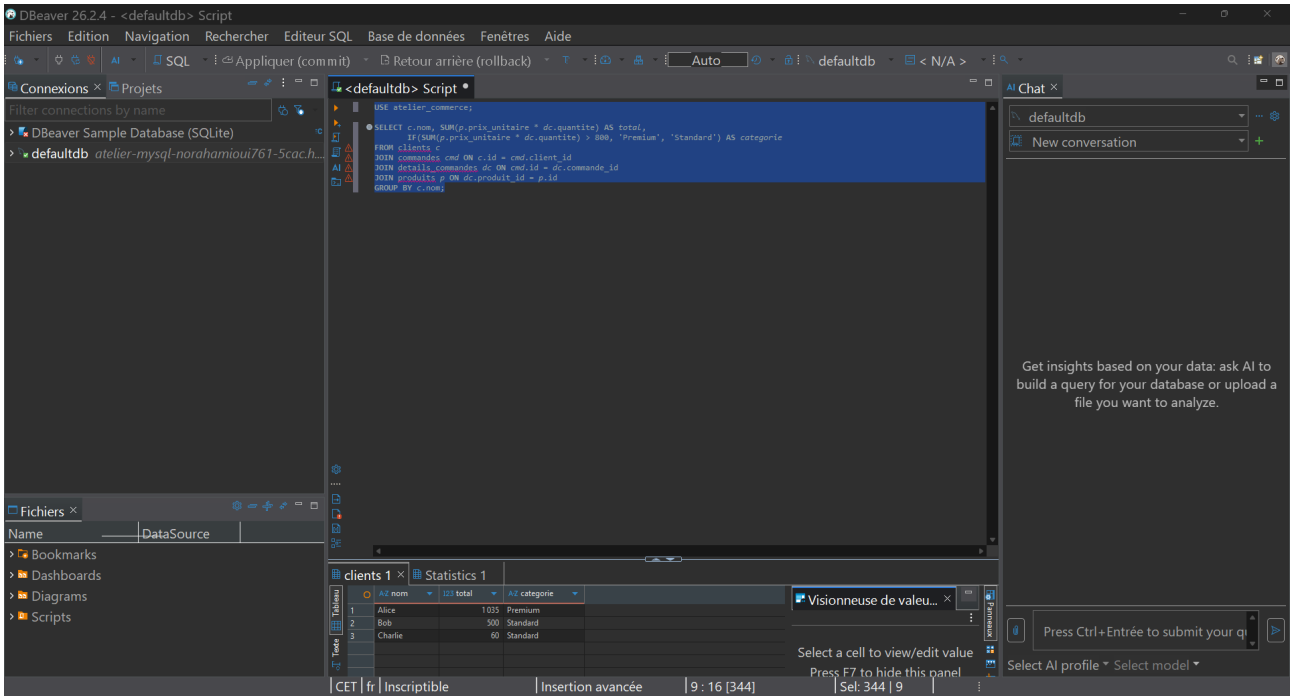


Figure 7 : classement des clients avec IF()

7. Index et EXPLAIN

Deux index ont été créés. Le résultat de EXPLAIN montre **type = ref** et **key = idx_nom_produit** : MySQL utilise bien l'index au lieu de parcourir toute la table.

```
CREATE INDEX idx_nom_produit ON produits(nom_produit);
CREATE INDEX idx_client_ville ON clients(ville);

EXPLAIN SELECT * FROM produits WHERE nom_produit = 'Souris';
```

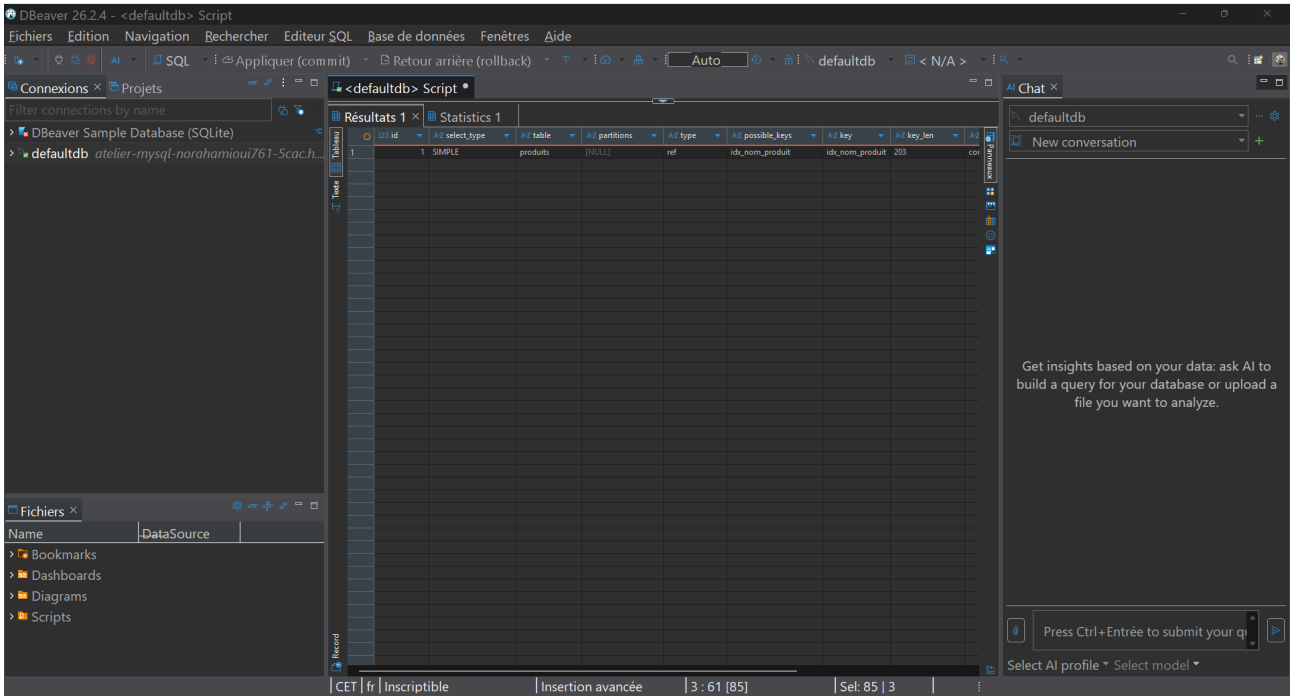


Figure 8 : EXPLAIN, utilisation de l'index idx_nom_produit

8. Vue vue_ventes

La vue regroupe clients, produits, montants et modes de paiement. La commande de Charlie n'a pas de paiement, d'où la valeur NULL (grâce au LEFT JOIN).

```
CREATE VIEW vue_ventes AS
SELECT c.nom, p.nom_produit, dc.quantite, (p.prix_unitaire * dc.quantite) AS montant, pa.mode_paiement
FROM clients c
JOIN commandes cmd ON c.id = cmd.client_id
JOIN details_commandes dc ON cmd.id = dc.commande_id
JOIN produits p ON dc.produit_id = p.id
LEFT JOIN paiements pa ON cmd.id = pa.commande_id;

SELECT * FROM vue_ventes;
```

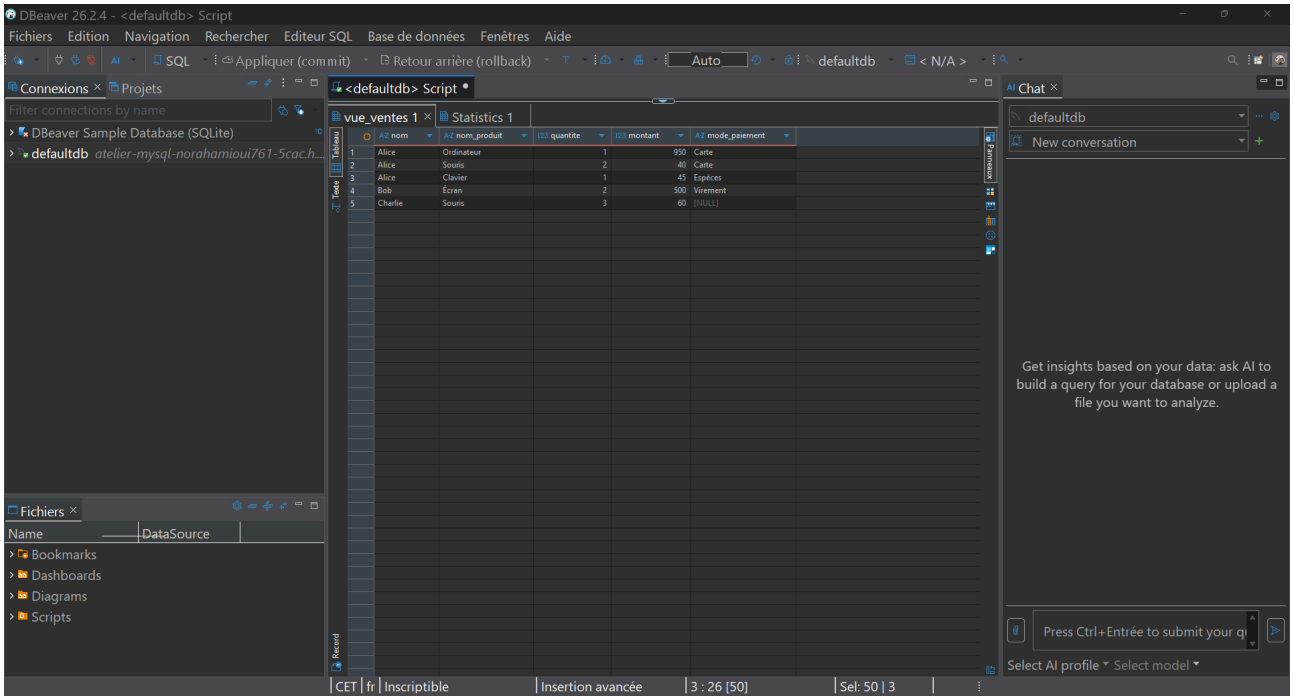


*Figure 9 : lecture de la vue (SELECT * FROM vue_ventes)*

9. Gestion des droits (GRANT, SHOW GRANTS, REVOKE)

Deux utilisateurs ont été créés. **manager** a tous les droits sur atelier_commerce. **vendeur** a reçu SELECT, INSERT, UPDATE, puis le droit UPDATE lui a été retiré avec REVOKE : il ne lui reste que SELECT et INSERT.

```
CREATE USER 'vendeur'@'localhost' IDENTIFIED BY '1234';
CREATE USER 'manager'@'localhost' IDENTIFIED BY 'admin';

GRANT SELECT, INSERT, UPDATE ON atelier_commerce.* TO 'vendeur'@'localhost';
GRANT ALL PRIVILEGES ON atelier_commerce.* TO 'manager'@'localhost';

SHOW GRANTS FOR 'manager'@'localhost';

REVOKE UPDATE ON atelier_commerce.* FROM 'vendeur'@'localhost';
SHOW GRANTS FOR 'vendeur'@'localhost';
```

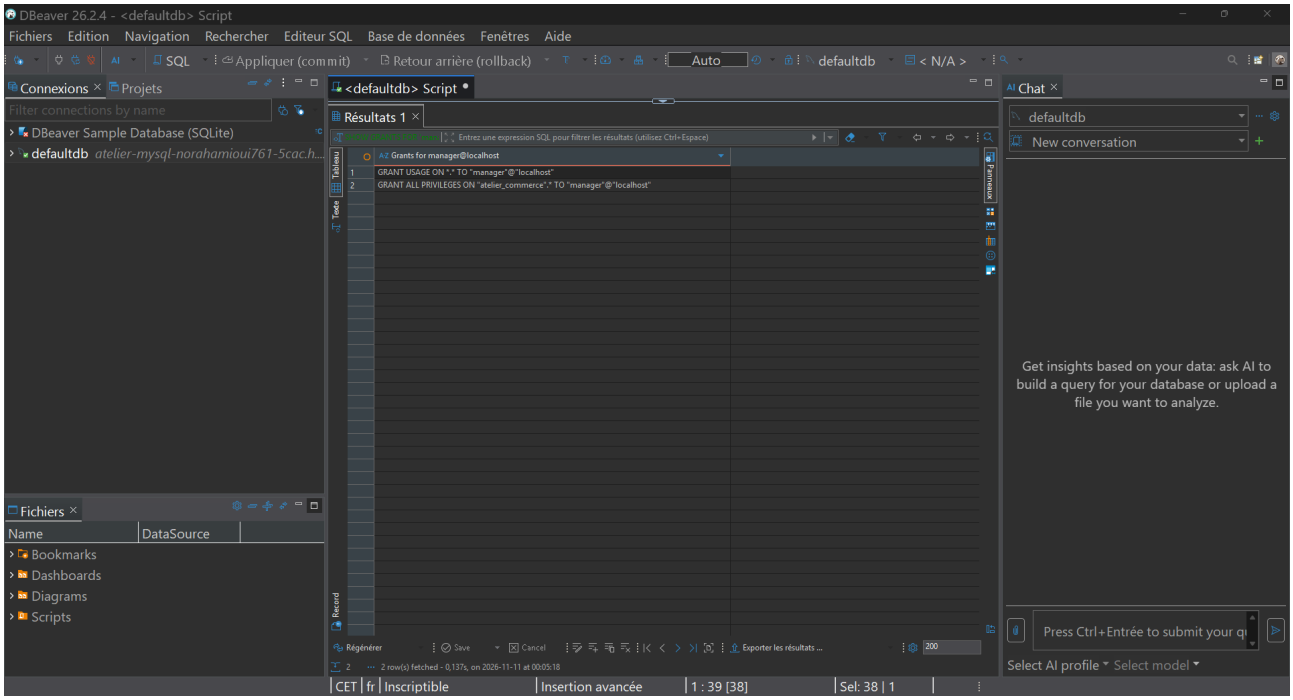


Figure 10 : SHOW GRANTS pour manager (ALL PRIVILEGES)

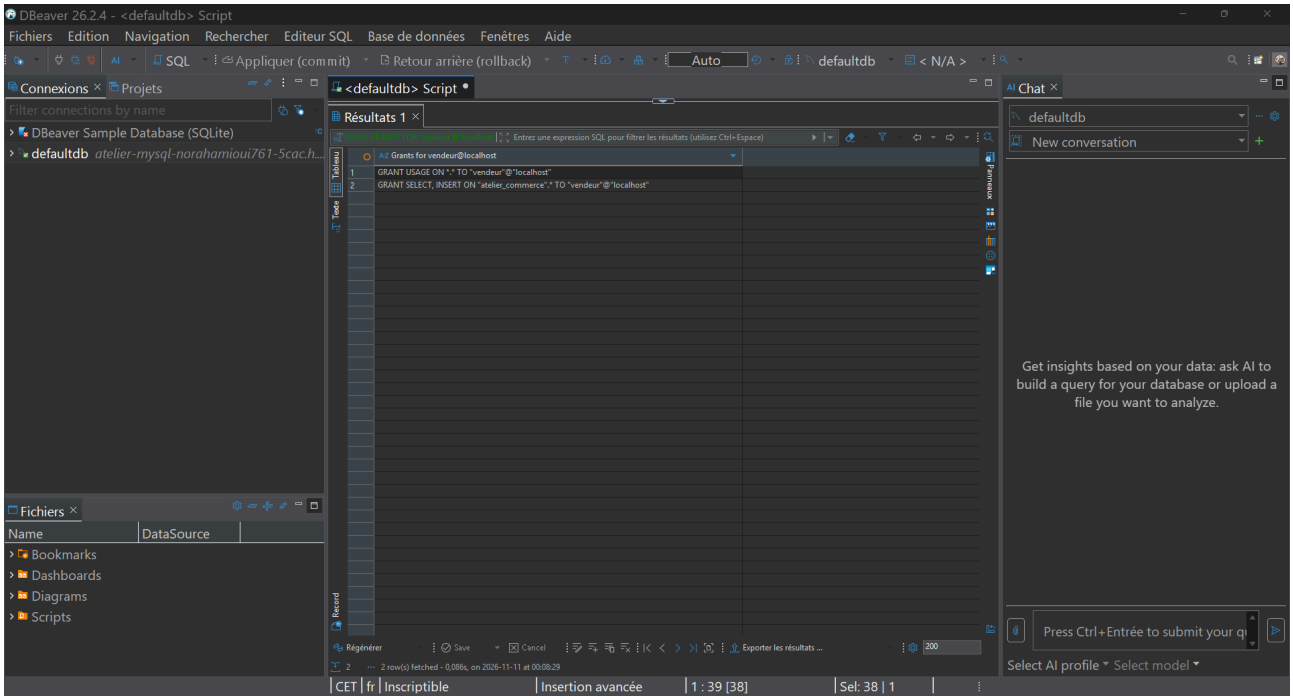


Figure 11 : SHOW GRANTS pour vendeur après REVOKE (SELECT, INSERT)